

Cogitator: Reproducible Agent-Mode Evaluation

George Gallagher

Abstract

Cogitator is a harness for deterministic, auditable evaluation of autonomous agents. It treats each evaluation as an experiment whose artefacts can be replayed off-line and independently verified. A domain-separated BLAKE3 hash chain commits to every causal event, producing a *witness root* that recovers identically when a run is replayed under identical inputs and environment assumptions. This paper describes the current version of Cogitator (v1.0.0) from first principles, including its commitment model, witness–provenance split, record/replay semantics, deterministic fault injection and verification workflow. New experimental results demonstrate witness–root invariance across thread counts and CPU pinning, drift localisation under deterministic chaos and tamper detection. A large-scale run on 200 000 cases illustrates run-level statistics. The paper concludes with related work, discussion of limitations and suggestions for future extensions.

Introduction

Benchmarking modern autonomous agents often reduces to a single aggregate score. Such numbers provide little insight into how an agent interacted with its tools or why it succeeded or failed. When a regression occurs, it can be unclear whether the root cause is a code change, drift in external services or nondeterministic behaviour. Cogitator addresses this reproducibility gap by treating evaluation as an experiment whose artefacts are replayable and auditable. For each run it emits a totally ordered trace of causal events and derives a *witness root*, a cryptographic commitment to the sequence and contents of those events. If a run is repeated with identical inputs and environment assumptions, Cogitator produces the same canonical artefacts and witness root; any mismatch localises differences to specific tool calls and fields.

Motivation.

Deterministic evaluation fosters transparency in research and accountability in industry. Publishing witness roots and artefact hashes alongside scores enables peers to replay experiments and understand the conditions under which a result was obtained. In industrial settings, deterministic traces simplify auditing, compliance and regression diagnosis, especially as agents interact with proprietary tools and models.

Outline.

Section [2](#) enumerates the goals and non-goals of Cogitator. Section [3](#) introduces the commitment model and key definitions. Section [4](#) discusses design decisions such as canonical JSON encoding, the choice of hash function and the witness–provenance split. The system overview in Section [5](#) describes artefact layouts and execution modes. Section [6](#) explains the verification workflow and gives a worked example. Section [7](#) presents empirical results. Sections [8–10](#) cover related work, discussion and limitations.

Goals and Non-Goals

Cogitator aims to make agent evaluations *deterministic*, *auditable*, *replayable* and *localisable*. With fixed prompts, seeds and tool configurations, the canonical event stream and witness root are identical across runs and hardware. Every causal event affecting behaviour—metadata, tool calls and agent decisions—is recorded in canonical JSON traces and tool transcripts. Recorded transcripts can be replayed off-line to reproduce agent decisions without contacting external services. When behaviour diverges, structured drift reports pinpoint the tool call and field responsible.

Cogitator is deliberately narrow in scope. It does not attempt to make inherently nondeterministic inference engines deterministic; it neither replaces agent frameworks nor defines benchmark specifications. It records provenance—timestamps, hostnames, thread counts and environment fingerprints—without committing it to the witness root. Provenance aids forensic analysis but does not influence evaluation outcomes.

Commitment Model

At the heart of Cogitator is a *domain-separated hash chain* built on the BLAKE3 function. Each event in the ordered event stream—metadata, tool call or agent decision—is canonically encoded to bytes using the JSON Canonicalisation Scheme (JCS). A new digest is computed by hashing the domain string (encoding the event type and index), the canonical bytes and the previous digest. The final digest is the *witness root*. By including the previous digest at each step, the chain commits to both the contents and the ordering of events: any alteration, reordering or truncation changes the final root. Provenance fields—timestamps, hostnames and thread counts—are recorded for auditability but excluded from the chain.

Key Definitions

the final digest of the domain-separated hash chain over all witnessed events. It commits to the sequence and contents of these events but not to provenance.

a projection of a tool transcript entry containing only the fields committed by the witness chain—tool name, request, response and success. It omits provenance-only fields such as latency or fault injection state.

a full record of each tool call, including the witness view and provenance (timestamps, fault events). Transcripts allow off-line replay and drift localisation.

a structured diff between recorded and replayed transcripts. It lists tool calls and fields where witness views diverged and annotates any injected faults. Because deterministic scheduling ensures that fault schedules are identical across runs, drift reports attribute divergences to agent or tool changes rather than random timing.

a digest of all files in a witness bundle directory. It is stored in `meta.json` and ensures that per-case artefacts (such as transcripts, witness views and drift reports) have not been altered.

Witness Views and Bundles

Each agent run produces a *witness bundle* containing the tool transcript, witness views, a drift report and per-artefact hashes. A bundle hash commits to all files in the bundle, allowing verifiers to detect forged transcripts. Recording provenance without committing to it reflects a conscious design choice: provenance aids forensic analysis but does not influence evaluation outcomes.

Threat Model

Cogitator assumes an adversary may tamper with artefacts, reorder events, truncate transcripts or substitute a run. The domain-separated hash chain defends against these threats. Any alteration of requests, responses or decisions changes the canonical encoding and therefore the witness root; verifiers recompute the chain and mismatches signal tampering. Because the chain includes the previous digest, removing or reordering events changes the final root. Bundle hashes cover all files in a witness bundle directory; a forger cannot provide a valid witness root without also reproducing the recorded transcript. Provenance fields capture environmental drift (library versions, environment variables and runtime versions); although these do not affect the witness chain, they aid forensic analysis.

Cogitator does not defend against an adversary who controls both the execution environment and the verification tool; integrating witness roots with signing and attestation frameworks is recommended for stronger supply-chain assurance.

Design Decisions

Cogitator's design balances security, performance and ease of verification. This section discusses key choices.

Canonical JSON Encoding

JSON's flexibility—unordered object keys, variable whitespace and multiple representations of numbers—makes it unsuitable for commitments without a canonicalisation scheme. Cogitator adopts the JSON Canonicalisation Scheme (JCS), defined in RFC 8785, which sorts object keys lexicographically, eliminates insignificant whitespace and encodes numbers and strings deterministically. JCS ensures that semantically identical JSON values have a unique byte representation so that hashing and comparison are unambiguous. This technique originates from security standards: the RFC notes that canonicalisation permits data to be exchanged in its original form while cryptographic operations performed on the canonicalised counterpart generate consistent results.

Choice of Hash Function

Cryptographic hash functions underpin the witness chain. Cogitator chooses BLAKE3 because it combines strong security guarantees with high performance and parallelism. BLAKE3 is significantly faster than SHA-2 and SHA-3, is secure against length extension attacks and can be parallelised across threads and SIMD lanes. Its tree structure allows incremental and streaming hashing without sacrificing speed and supports extendable output, key derivation and message authentication. These properties also underpin the pseudorandom number generator used for fault injection. SHA-2 and SHA-3 provide similar security but are slower and lack built-in extendable output.

Domain-Separated Hash Chain

Merkle trees are a classic approach to committing to a collection of values; they provide efficient inclusion proofs and are widely used in blockchains. Cogitator instead adopts a linear, domain-separated hash chain. The chain commits to the total order of events, which is essential for deterministic agent runs: reordering two tool calls must change the witness root. A Merkle tree could commit to a set of events without order, requiring additional mechanisms to enforce sequencing. Because the primary use case is comparing entire runs rather than proving inclusion of a subset, the linear chain simplifies verification and reduces per-event overhead. Extending Cogitator with Merkle-tree commitments for selective disclosure is left for future work.

Witness–Provenance Split

Separating witnessed semantics from provenance is inspired by functional package managers such as Nix and Guix. Witnesses represent the semantics of an evaluation: the inputs, decisions and outputs that determine scores. Provenance captures contextual information that does not influence the outcome but aids debugging—timestamps, hostnames, thread counts, environment fingerprints and simulated latency. Recording provenance without committing to it allows runs to be replayed even when environment details differ; provenance remains available for forensic analysis.

Record/Replay versus Full Execution Capture

Traditional debugging tools such as *rr* record low-level instructions and system calls to allow exact execution replay. While powerful for debugging, such full execution capture is heavyweight and tightly coupled to the operating system. Cogitator instead records tool-call transcripts at the agent–tool boundary. This coarse granularity omits internal nondeterminism within tools and the language runtime but dramatically reduces overhead and makes transcripts portable across machines. Replay uses the recorded transcript as a black-box proxy for the tool. This trade-off sacrifices the ability to inspect arbitrary program state in exchange for portability and drift localisation.

Deterministic Fault Injection

Deterministic simulation testing introduces failures at tool boundaries to surface bugs that only appear under adverse conditions. Cogitator designs fault injection around a pseudorandom number generator derived from BLAKE3, seeded by the run seed, step index and a user-configured fault profile. At each tool call the generator draws an integer from the range $[0, 10^6)$; if the result is below the configured per-million rate, a fault is applied. Four fault types are supported: *Timeout*, producing a stable error outcome and a simulated delay; *Drop*, omitting the response entirely; *Corrupt*, applying deterministic bit flips or value modifications to the response; and *LatencySim*, introducing deterministic latency recorded only in provenance. Because the generator and schedule depend solely on the run seed and step index, the fault schedule is replayable and identical across machines, contrasting with chaotic testing where faults are injected nondeterministically.

System Overview

Cogitator consists of two execution modes and a collection of command-line utilities. The *core* mode emits a canonical trace of events and computes a witness root. The *agent* mode adds an agent trace, a tool transcript, a drift report and per-run witness bundles. Outputs are

organised into a run directory for global artefacts and per-case subdirectories for agent artefacts. A typical layout includes:

- `meta.json` – stores the run seed, configuration, expected witness root and per-artefact digests.
- `trace.jsonl` – contains the canonical event stream as a JSON lines file.
- `results.csv` – summarises per-case scores for larger runs.
- `summary.json` – summarises high-level statistics.
- `analysis.json` – stores derived metrics and additional analyses.
- `witness_manifest.json` – lists artefacts and their BLAKE3 digests; the bundle hash is a digest over the ordered list of (name, hash) pairs.
- `run_0000/`, `run_0001/`, ... – per-case directories containing the tool transcript, witness views, drift report and optional provenance files such as Nix metadata.

Command-line interface.

The Cogitator CLI exposes subcommands to run experiments and verify artefacts. The `run` command evaluates a specified agent on a set of cases with flags to control the number of cases, random seed, thread count, fault profile, output directory and whether to record transcripts. The `verify` command replays recorded transcripts and recomputes witness roots, checking them against `meta.json`. The `replay` command runs an agent in replay mode using the recorded transcripts, producing drift reports where behaviour diverges. Additional commands list available fault profiles, compute bundle hashes and clean run directories. These subcommands are invoked by the CI gate described below.

Benefits on NixOS

Cogitator functions correctly on any modern Linux, macOS or Windows platform, but running it inside a purely functional environment like NixOS offers additional reproducibility benefits. NixOS builds software from declarative specifications that pin every dependency, ensuring that a given revision of Cogitator always uses the same compiler, libraries and toolchain. The provided `flake.nix` and `shell.nix` files define a development environment that installs Rust toolchain components (`cargo`, `rustc`, `rustfmt`) and locale data. When entering this environment via `nix develop` or `nixshell`, the `SOURCE_DATE_EPOCH` variable is set to a constant so that timestamps written into provenance fields are deterministic. Locale archives are likewise fixed, preventing hidden nondeterminism from locale or collation differences.

During a run, Cogitator may optionally collect Nix provenance via the `--nix-provenance=auto|on|off` flag. When enabled, the runtime records the Nix and NixOS versions, flake metadata, current system derivation and whether the Nix store was detected. This information is stored in `nix_provenance.json` as provenance only; it

never influences the witness root. Recording Nix metadata provides context for reproducing evaluations on machines with identical Nix environments while maintaining witness stability.

NixOS also complements Cogitator’s design goals of reproducibility and isolation. Functional package managers such as Nix and Guix eliminate impurity by ensuring builds have no side effects and can be recreated anywhere. When Cogitator is used inside a Nix shell, dependencies do not leak into the host system and version upgrades are explicit. This synergy reduces the risk of environmental drift across evaluation runs and improves the portability of witness bundles. Operators who wish to reproduce evaluation artefacts years later can simply check out the repository, run `nix develop`, and invoke the same `cogitator run` commands to obtain identical witness roots.

Verification Workflow

Cogitator defines two levels of verification.

Meta+Trace verification.

A verifier replays the canonical trace (`trace.jsonl`) under the same deterministic scheduling and recomputes the witness root from scratch. The recomputed root is compared against the expected root recorded in `meta.json`; any discrepancy indicates that either the trace has been tampered with or the implementation is nondeterministic. Meta files also include per-artefact hashes and the run configuration, enabling verifiers to detect missing or altered files.

Witness bundle verification.

Each agent run (a “case”) produces a subdirectory containing the tool transcript, agent decisions, drift report (if any) and witness root. A witness bundle includes both the transcript and a witness view of each tool call—only the fields committed to the witness root. When verifying a bundle, a third party recomputes the hash chain over witness views and compares it against the stored root. If a mismatch is found, the verifier can locate the exact event whose witness view differs. Because transcripts include uncommitted provenance (latency, simulated faults, hostnames), mismatches in those fields do not invalidate the witness root but may inform drift diagnosis.

CI integration.

Projects integrate Cogitator into continuous integration (CI) by pinning expected witness roots and running `cogitator run` followed by `verify` on each commit. A job fails if any expected root changes or if verification reports drift. This gate, called the *Ordeal*, ensures that regressions are detected before deployment. The CI configuration can pin specific versions of agents and tool adapters, and override environment variables (e.g., `SOURCE_DATE_EPOCH=315,532,800` representing 1980-01-01 UTC) to ensure reproducibility.

Worked Example

To illustrate how Cogitator constructs witness roots and detects drift, consider a toy evaluation consisting of a single agent decision and one tool call. The run begins with a metadata event recording the run seed and configuration. This event is canonically encoded via JCS and hashed with a domain string (e.g., `meta:0`) to produce the first digest. The next event is a tool call recording the request and response. This event is encoded via JCS, concatenated with the previous digest and hashed with domain string (e.g., `call:1`). Finally, the decision event records the chosen action and score; after hashing this event with domain string (e.g., `decision:2`) and the previous a digest, we obtain the witness root. The run produces a `trace.jsonl` file containing these events, a `meta.json` with the expected witness root and a `run_0000` directory with the transcript.

Suppose a future change modifies the agent to omit the article “the” in its request. During a regression run the agent sends `"text": "sky was clear and blue."` instead of the original `"text": "The sky was clear and blue."`. Because the request string is part of the witness view, its canonical encoding differs. In replay mode Cogitator consumes the recorded transcript and expects the original request. It detects a mismatch at the tool call and emits a drift report identifying the event, tool call step, witness chain diverges at this event and the drift report localises the difference to the specific tool call. Because the witness chain diverges at this event, the replayed witness root differs from the recorded root and verification fails.

Fault Injection Example

Deterministic fault injection can reveal bugs that only manifest under adverse conditions. In the same scenario, configure a timeout fault rate of two in a million. During live execution the pseudorandom generator draws values at each tool call. Suppose the generator draws 1,234 for the first call, which is below the threshold; a *Timeout* fault is injected. The tool call event in the transcript records the request, fault type and absence of a response. The agent receives a deterministic error and may choose to retry or abort. The witness view for this call commits only to the fault type and absence of a response; the delay is recorded in provenance but excluded from the witness chain. In replay mode the same pseudorandom draw is recomputed (because it depends on the run seed and step index), so the fault schedule is identical. If a subsequent code change alters error handling—for example, the agent attempts a different fallback—the witness view changes and the drift report localises the difference to the decision event.

Experimental Evaluation

We evaluate Cogitator along three axes: (1) determinism across thread counts and CPU affinity, (2) drift detection under deterministic chaos and (3) overhead of canonicalisation and hashing. All experiments were conducted using Cogitator v0.1.0 on commodity x86–64 hosts.

Determinism across thread counts and affinity

The first experiment assesses whether witness roots remain invariant across different levels of parallelism and under explicit CPU pinning. We executed the same workflow on a machine with `RAYON_NUM_THREADS` $\in \{1, 2, 4, 8, 16\}$ and also under `taskset` pinning to cores $\{1, 3, 7, 13\}$. In all cases the witness root was identical; the full 64-hex root is

804387dc9676ae4703aea2f48d9737b5a0b28ac26b7b9cb381bba5da0fd0ec42, demonstrating that parallel scheduling and CPU affinity do not affect the recorded event stream. Table 1 summarises the results.

Witness–root invariance across thread counts and CPU pinning.

Configuration	Witness root (truncated 16 hex)	Observations
1 thread	804387dc9676ae47	identical root
2 threads	804387dc9676ae47	identical root
4 threads	804387dc9676ae47	identical root
8 threads	804387dc9676ae47	identical root
16 threads	804387dc9676ae47	identical root
pinned cores {1,3,7,13}	804387dc9676ae47	identical root

Drift detection under deterministic chaos

We next demonstrate how Cogitator localises drift when failures are injected deterministically. Using a fault profile that injects timeouts and drops at a 2 agent: a baseline and a regressed variant that subtly altered a tool request. Aggregate metrics (pass rates and scores) were identical, but Cogitator’s drift report flagged a divergence at tool call 3. Because the fault schedules are deterministic, the injected timeout occurred at the same step in both runs; thus the divergence is attributable to the agent change rather than random timing. A second pair of runs with fault injection disabled reproduced the same divergence, confirming that deterministic chaos neither hides nor amplifies agent regressions.

Overhead of canonicalisation and hashing

To quantify overhead we compared runtime when writing artefacts with and without canonicalisation and witness hashing. As in prior work, overhead remained modest. For artefact sizes of 10 KB, 100 KB and 1 MB the additional cost of canonicalisation was under 5 overhead is further amortised when runs contain many tool calls.

Large-scale evaluation statistics

Cogitator emits a `results.csv` summarising per-case scores. We analysed a run of 200 000 cases (each a summarisation task). The pass rate is defined as the fraction of cases whose score is at least 0.8. Table 2 summarises the results: only 27.83% passed while 144 340 failed. These aggregates provide a high-level overview of agent performance while preserving the ability to audit individual cases via transcripts and witness views.

Summary statistics for a 200k-case evaluation.

Statistic	Value
Total cases	200 000
Passed	55 660
Failed	144 340
Pass rate	27.83 %
Mean score	0.362
Median score	<i>n/a</i>

Drift and tamper evidence examples

Cogitator detects subtle tampering. In one experiment the field `meta.witnessed.executed_runs` was incremented by 1 in `meta.json`. Verification recomputed the witness root and found a mismatch: the expected root `804387dc...ec42` differed from the computed root `643edc9b...3c50`. Another experiment edited the `trace.jsonl` semantic ordering: swapping two events triggered a trace ordering violation at line 2 (tuple 0, 1 after 0, 1). These examples illustrate how domain-separated hash chains and manifest hashes detect tampering without bespoke auditors.

Baseline vs. regressed drift demonstration.

We also compared baseline and regressed runs under a synthetic fault profile (`seed=42`, `fault_profile=stress`, `threads=8`). The baseline produced a witness root `60f16b0ec05d52e2984a820d5ecd7e49f6d0774b4ecd9a72927d3b6342338335` and no drift issues. The regressed variant produced root `8072943ac531413cd7b508c2331d0f4fe4236cd6420be0b02a2384a2d57ca750` with two drift issues. Under fault injection these roots changed to `50a3e38bdf16b3eb1174b2e36b61df3f905f89dee4727e4d8c81dfd6ae065435` and `42f66495f59236290074999e692f98cc1eb45a444bec8f747b9ae4c0a6079520`, respectively, with zero and three drift issues. Cogitator's drift reports localised the divergences to specific tool calls and fields.

Related Work

Cogitator draws on ideas from several domains. Functional package managers such as Nix and Guix record dependency graphs and support reproducible environments, storing packages in immutable directories named by cryptographic hashes of their dependencies. Cogitator borrows the principle of separating committed semantics from environment provenance, enabling stable commitments while still recording context. Record/replay systems like *rr* capture low-level execution streams for debugging and reverse execution. Cogitator operates at a higher level, recording tool-call transcripts rather than instruction streams and trading completeness for portability and clearer drift localisation. Deterministic simulation has been used to test distributed systems under many failure scenarios. FoundationDB’s simulator conducts deterministic simulations of an entire cluster within a single process; determinism allows perfect repeatability and facilitates controlled experiments. The Redstone framework intercepts system calls and bases program flow on a pre-chosen seed so that failure conditions can be reliably reproduced. Cogitator adapts this strategy to agent evaluation by injecting deterministic faults at tool boundaries. Supply-chain attestation frameworks such as in-toto attach cryptographic signatures and metadata to each phase of the build process to ensure end-to-end integrity. In-toto allows users to verify that software supply chains have not been tampered with. Cogitator is narrower in scope: it focuses on deterministic traces and witness roots that commit to an agent run’s causal stream, plus drift reports for regression diagnosis. Experiment tracking tools such as MLflow provide APIs and user interfaces for logging parameters, code versions, metrics and artefacts. Cogitator complements such frameworks by focusing on deterministic evaluation harnesses and cryptographic commitments, enabling independent verification and drift localisation.

Discussion and Outlook

Deterministic evaluation bridges the gap between reproducible software engineering and modern AI benchmarking. By committing to witnessed semantics and separating provenance, Cogitator offers strong guarantees about what was executed without obscuring the context in which it ran. In research settings this facilitates transparent reporting: witness roots and artefact hashes published alongside scores enable peers to verify and replay runs. In industrial settings deterministic traces simplify auditing, compliance and regression diagnosis.

Looking ahead, witness roots could become part of a broader ecosystem of attestations. Build systems might attach witness roots to machine-learning models and data pipelines, creating an auditable chain from training to deployment. Evaluation harnesses could integrate confidential computing or secure enclaves, proving that sensitive data was processed under fixed code. Standardisation efforts may define common event schemas and domain separators, enabling interoperation between evaluation frameworks. Coupling deterministic evaluation with open-source tool adapters and benchmarks could democratise agent research by lowering the barrier to verifiable experimentation.

Limitations and Future Work

Cogitator does not attempt to make inherently nondeterministic inference engines deterministic. Instead it isolates stochastic components behind transcripts and emphasises replayability. In live mode tool calls (including large-language-model calls) are recorded; in replay mode recorded outputs are reused verbatim. This makes runs auditable and replayable but does not guarantee that a third party can reproduce live external behaviour without the transcript.

The current hash chain does not support efficient inclusion proofs; if partial disclosure or selective verification becomes a primary requirement, Merkle-tree commitments are a plausible extension. Richer integrations with real tool adapters and case studies remain ongoing work. Because transcripts record entire tool requests and responses, they may contain sensitive user data or personally identifiable information. Future work could explore witness schemes that commit to hashed summaries of tool responses while allowing selective disclosure of sensitive fields. Domain-separated hash chains commit to the order and contents of events, which is ideal for reproducibility but may expose the full evaluation sequence to verifiers; Merkle-tree-based commitments might allow zero-knowledge proofs of high-level properties without revealing every intermediate state. Finally, Cogitator focuses on deterministic scheduling at the tool-call boundary; it does not capture internal nondeterminism within agent code or tools (e.g., thread races or floating-point rounding). This keeps overhead low but places the burden on tool developers to ensure deterministic behaviour.

Conclusion

Cogitator offers a principled approach to deterministic agent evaluation. By committing to witnessed semantics, separating provenance, recording transcripts and injecting deterministic faults, it enables auditable, replayable and localisable experiments. Empirical results confirm that witness roots are invariant across parallelism and CPU pinning, that deterministic chaos surfaces regressions without masking them and that canonicalisation and hashing incur modest overhead. Large-scale runs demonstrate how run-level statistics can be summarised without sacrificing traceability. Together these properties position Cogitator as a building block for reproducible AI research and robust continuous evaluation pipelines.

Addendum: Release v1.0 Updates

Artifact serialization

Cogitator continues to serialize witnessed artefacts using the strict canonical JSON subset adopted in earlier versions. Only integers are permitted as JSON numbers for canonical traces and other witnessed artefacts; the ordering and formatting remain RFC 8785 compliant. By contrast, report or display artefacts such as *results.json*, *summary.json* and *analysis.json* now use standard JSON serialization. These files may contain floating-point numbers and are explicitly marked as non-witnessed: they are not part of the witness root and may evolve without affecting verification. Because JSON numbers cannot express non-finite values such as NaN or Infinity (see ECMA-404 / RFC 8259), Cogitator does not emit such values in report metrics. Any future metrics that might produce non-finite values should either be clamped to finite numbers or clearly signposted as exceptional.

Determinism evidence

To check that witness roots remain unchanged across thread counts on Windows, we ran a regression test with seed = 42 over 1,000 runs. With `RAYON_NUM_THREADS = 1` the witness root was `82f6123b33295406a22e15b1d6c121411aa59d9fa817eef77ca3220378278792`. Setting `RAYON_NUM_THREADS = 16` produced the same value, and a binary comparison (`fc out1\witness_root.txt out2\witness_root.txt`) showed no differences. This is a sanity check for this workload and configuration rather than a proof of global determinism. Rayon exposes the `RAYON_NUM_THREADS` environment variable to control the number of worker threads. Cogitator records the thread count in provenance but excludes it from the witness commitment.

Stability contract for 1.0

Version 1.0 formalises Cogitator's public API. The stable contract includes:

- CLI subcommands and key flags for run and verify.
- Witness semantics: what contributes to the witness root and how verification behaves.
- On-disk witnessed formats: the *meta.json* schema version, the structure of *trace.jsonl* required for *verify*, and the meaning of *witness_root.txt*.

The following are not part of the stable contract and may evolve without a breaking change:

- Report artefact formats (*results.json*, *summary.json*, *analysis.json*).
- CSV layout, pretty-printing and TUI/UI presentation.
- Provenance and reporting fields that do not affect witness semantics.

Following Semantic Versioning, version 1.0.0 defines Cogitator's public contract.

Backward-compatible additions increase the minor version, bug fixes bump the patch version, and changes that alter witness verification behaviour or the schemas of witnessed artefacts require a major version increment.